

Avoid using vulnerable

components: In many places, developers are free to choose the open source components and integrate them in their applications. In doing so, what often gets ignored is whether the particular version of the chosen component has any reported vulnerabilities. A suggested way to safeguard applications is to prevent using vulnerable components right from the start when building new products. Therefore:

- a) It is important that when you pull new dependencies, pick them only from official sources through secure channels.
- b) Check online resources to see if there are any published vulnerabilities. However, the information about vulnerabilities might be hard to find at one place, which makes it difficult for developers to detect components with flaws.
- c) One way of ensuring the above and regulating the undesired usage could be to disable the direct access to public repositories and build your private central repository of only approved components. The applications can pull dependencies only from this private repo, and any new dependency addition must go through a security review and approval process.

Identify affected components:

There is a possibility that a component was carefully chosen, but a vulnerability got identified and reported sometime later. Also, there would be existing and legacy applications that were built when this system was not in place. So, applications must be continuously scanned to know if the open source components that are already a part of your inventory and products contain any vulnerabilities. This can be automated through the use of various commercial or open source tools. Many of these support a wide range of languages and help not just identify but also categorise the vulnerabilities as per the severity.

Prioritise and fix:

With thousands of new vulnerabilities disclosed every year, the scan report will give you a long list of issues to fix, especially when you scan your projects for the first time. You need to understand the severity of vulnerabilities to see which ones are the most critical for you. Many tools help classify issues using established methods and parameters like CVSS, CWE, etc. Issues that have high severity scores, easy exploitability and an available fix could be addressed first. Besides this, one must factor in the business impact of a vulnerability such as the possibility of an SQL injection, any kind of financial data threat, PII (personally identifiable information) leakage or IP loss, and prioritise accordingly. Each application is different and so is the associated risk.

As these are publicly disclosed vulnerabilities, the fixes usually involve an upgrade to a higher version of the dependency. They will be easier to fix if upgrades don't break the existing functionality, but sometimes there could be a major version upgrade which might require bigger changes. Similarly, some fixes may require customers to download a new release, and you may not want to bother customers with too frequent updates. And sometimes the affected components might no longer be supported or have reached end of life, which will require switching to an alternative component that is actively developed or build one in-house. Taking all of this into account, one can prioritise in a way that there is balance

between the pace of development and protection against critical issues.

Build pipeline integration —

shift left: Implementing measures at the right stage of the product development life cycle is important too. Finding and fixing at a very late stage can cause significant delays in release and increase the project cost. Most of us already have CI/CD pipelines in place to automate the build, test and deployment process. These pipelines can be leveraged to plug vulnerability detection and accelerate automated scanning, find issues early and often, and control the process from a single place. Build-wise scan reports are also available, and builds with critical vulnerabilities can be flagged or even blocked. There can be IDE integrated scans, too, to get an early feedback.

Continuous monitoring: While some projects remain in active development and get continually scanned at the CI/CD security gate, those less updated might have latent vulnerabilities that remain unidentified for a long time. By having a process to run periodic scans on repositories we can constantly check for new vulnerabilities and contain the risk.

By implementing a good open source management strategy, which ensures continuous monitoring and remediation that pervades the entire development life cycle, we can optimise the benefits of open source and accelerate software development while minimising the potential risks. **END** 🐧

References

- [1] <https://resources.github.com/whitepapers/Open-source-enterprise-innovation/>
- [2] <https://owasp.org/www-project-top-ten/>

By: Minni Arora

The author works as associate vice president architect with Naukri.com. She has fourteen years of experience in architecture, design and development of Web and mobile applications.